Digital Object Identifier 10.1109/ACCESS.2017.DOI

Graph Retrieval-Augmented Generation for Improving Retrieval Precision and Syntactic Validity in Kubernetes Configuration

JIHAN AURELIA[1], DIMITRI MAHAYANA[1], and NAUFAL IRSYADI[1]
[1]School of Electrical Engineering and Informatics, Institut Teknologi Bandung, Bandung 40132, Indonesia
Corresponding author: Jihan Aurelia (e-mail: 18222001@std.stei.itb.ac.id).

ABSTRACT Kubernetes has become the de facto standard for container orchestration, yet the complexity of its YAML configuration drives the use of Large Language Models (LLMs) that are prone to producing semantically incorrect configurations. Vector-based retrieval-augmented generation (RAG) is insufficient because it fails to capture the hierarchical relations among Kubernetes components. This work builds a graph retrieval-augmented generation (GraphRAG) system that derives a knowledge graph (KG) deterministically from the Kubernetes OpenAPI specification, traverses it with a depth that adapts to query intent, and validates generated YAML manifests directly against the graph without requiring a live cluster. The system is evaluated against Vector RAG and a context-free LLM using 102 practitioner-validated test scenarios. Results show a clear advantage on the retrieval dimension: an F1 score of 0.71 versus 0.24 for Vector RAG, a statistically significant difference ($p<0.001$). Graph traversal also successfully covers most of the expected schema relations, reaching a hop accuracy of 0.76. In contrast, answer text quality does not differ significantly across systems, indicating that the knowledge graph's contribution lies in the traceability and completeness of structural retrieval rather than in language fluency. These findings empirically map the dimensions where graph representation provides a genuine advantage in the Kubernetes configuration domain.

INDEX TERMS graph retrieval-augmented generation, intent-adaptive depth traversal, knowledge graph, Kubernetes, LLM hallucination, YAML validation

I. INTRODUCTION

KUBERNETES [1] has established itself as the de facto industry standard for container orchestration, driven by a mature ecosystem and the flexibility of declarative infrastructure management [2], [3]. However, this declarative paradigm requires developers to write verbose, hierarchical YAML configurations with strong inter-object dependencies that are frequently left implicit [4]. As a consequence of this complexity, roughly 80% of Kubernetes operational incidents are rooted in this configuration complexity, and 18% of open-source configuration files have been found to contain serious security vulnerabilities [5].

This situation has driven the use of LLMs to help compose and explain Kubernetes configurations. However, LLMs can generate code that appears syntactically correct yet is semantically wrong [6], a common manifestation of the broader hallucination tendency of generative models [7]. Mitigation efforts through vector-based retrieval-augmented generation (RAG) also fall short: such approaches rely solely on embedding similarity and therefore fail to capture the causal and hierarchical relations that form the core logic of the Kubernetes domain [8].

The graph retrieval-augmented generation (GraphRAG) literature addresses part of this limitation by injecting structured knowledge representations into the retrieval process [9]. However, as detailed in §II, two technical gaps persist: existing knowledge-graph construction pipelines are largely LLM-driven and therefore difficult to reproduce across runs, and existing traversal mechanisms typically apply a fixed depth uniformly across all query types despite markedly different context needs.

Based on these two gaps, this work proposes three technical contributions. First, a deterministic knowledge-graph construction pipeline based on the OpenAPI schema, so that

extraction results are consistent across runs. Second, a traversal mechanism whose depth adapts to the query intent type (intent-adaptive depth traversal), rather than a uniform fixed depth. Third, a three-layer, KG-grounded structural YAML validation mechanism that serves as a static alternative to a dry-run against a live Kubernetes cluster. The remainder of this paper reviews related work (§II), presents the system design (§III), describes the evaluation setup (§IV), reports results and discussion (§V), and concludes (§VI).

II. RELATED WORK

A. RAG AND LLM HALLUCINATION IN TECHNICAL DOMAINS

Retrieval-augmented generation (RAG) has become a standard approach for grounding LLM output in external knowledge, reducing hallucination by conditioning generation on retrieved context [10]. RAG pipelines built on generic chunking and vector similarity nevertheless remain vulnerable to weak semantic boundaries between retrieved passages, which has motivated chunking strategies that better align retrieval units with document structure [11]. Evaluation frameworks such as RAGAS [12] formalize this concern by decomposing answer quality into sub-metrics, including faithfulness (the traceability of generated claims to retrieved context) and answer relevance, both of which this work adopts (§IV). Hallucination itself remains an open problem for generative models in general [7], and is particularly consequential for code and configuration generation, where syntactically plausible output can still be semantically wrong [6]. Together, these findings motivate retrieval mechanisms with an explicit, inspectable structure rather than opaque similarity search alone.

B. KNOWLEDGE GRAPH CONSTRUCTION: LLM-BASED VS. SCHEMA-DERIVED

Constructing the knowledge graph itself is an open research question. A substantial body of work uses an LLM to extract entities and relations directly from unstructured text [13], [14], and surveys of graph construction techniques document a wide range of such extraction pipelines [15]. Because LLM-based extraction is inherently stochastic, however, the resulting graph structure varies across runs, which complicates reproducibility and auditability, a concern also raised more broadly in domain-specific knowledge-graph construction [16]. Schema-derived construction, in which a graph is built by explicit, deterministic transformation rules applied to a formal specification rather than by model inference, offers an alternative that avoids this variability, but it has received comparatively little attention for API-defined domains such as Kubernetes.

C. GRAPHRAG AND TRAVERSAL STRATEGIES

GraphRAG systems combine graph traversal with LLM generation to answer questions grounded in structured knowledge. G-Retriever [17] retrieves relevant subgraphs for textual graph question answering; HSG-RAG [18] constructs a hierarchical knowledge base for embedded system development; a GraphRAG-enhanced dialogue engine has been applied to a civil IoT platform [19]; and NeuSym-RAG [20] combines neural and symbolic retrieval for structured document question answering. Across these systems, graph traversal typically proceeds to a fixed depth regardless of query type, an assumption inherited from general-purpose GraphRAG formulations [9]. Applied specifically to Kubernetes documentation, hybrid retrieval-augmented generation has been shown to outperform purely vector-based retrieval by incorporating structured domain knowledge [8], but without varying traversal depth by query intent and without validating generated output against the graph itself. This work addresses both gaps directly.

III. SYSTEM DESIGN

Fig. 1 summarizes the architecture of the GraphRAG system, spanning an offline knowledge-graph construction zone and a runtime retrieval-and-generation zone.

A. DETERMINISTIC KNOWLEDGE GRAPH CONSTRUCTION

The knowledge graph is built directly from the *definitions* block of the Kubernetes OpenAPI specification (`swagger.json`) through explicit transformation rules, yielding 725 nodes and 18 edge types across 7 semantic relation categories (structural, workload, storage, configuration, networking, RBAC, and autoscaling). Graph construction uses two complementary strategies: a deterministic strategy that converts every `$ref`, `allOf`, `oneOf`, and `anyOf` reference directly into a structural edge, and a semi-deterministic strategy that forms domain-relevant operational relations that are not explicitly stated in the schema, such as the relation between *Service* and *Pod*. Because every transformation rule is explicit and executed without any stochastic component, the resulting graph is consistent across runs and every edge can be traced back to its originating schema reference, unlike LLM-based extraction approaches. The semantic vector representation of each node is stored directly as a graph property in Neo4j, so similarity-based search can be executed within the same graph without a separate vector store.

B. RETRIEVAL WITH ADAPTIVE DEPTH

The retriever module implements a three-stage cascading retrieval process. The first stage searches for a node whose name exactly matches the entity extracted from the query intent and uses it as the seed node; this gives the highest precision because it relies on exact matching rather than semantic approximation. The second stage, executed only if the first stage fails, searches for the node with the highest vector similarity to the query as a fallback. The third stage recursively traverses the graph from the seed node with a depth limit that adapts to the query intent type: a depth of 2 for the *explain* and *followup* intents, which are resource-specific, and a depth of 3 for the *generate_yaml*, *trace_relationship*, and

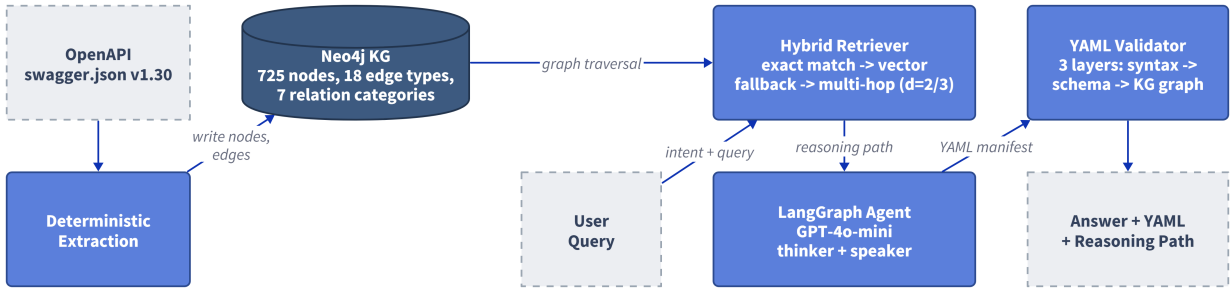

FIGURE 1. GraphRAG system architecture: deterministic knowledge-graph construction from `swagger.json` (left), and the retrieval and answer-generation flow with graph-grounded YAML validation (right).

planning intents, which require broader relational context. This bound was chosen because nodes beyond depth 3 are dominated by generic types shared by dozens to hundreds of resources, which lowers context precision without adding relevant information. The relational path traversed is assembled into a reasoning path and injected into the prompt as verifiable evidence for the user.

C. THREE-LAYER KNOWLEDGE-GRAPH-GROUNDED YAML VALIDATION

When a query's intent is classified as *generate_yaml*, the resulting YAML manifest passes through three sequential validation layers: the first layer checks syntactic correctness using a standard YAML parser; the second layer checks structural compliance against the official Kubernetes schema; and the third layer queries the knowledge graph directly to verify that all required fields are present for each resource. This third layer is what makes the validation KG-grounded: required-field verification is performed without needing a dry-run against a live Kubernetes cluster, because the same schema information is already available in the graph built during construction.

All of the above mechanisms run on a LangGraph pipeline in which GPT-4o-mini plays a dual role as *thinker* (intent classification and entity extraction) and *speaker* (answer generation), with Neo4j serving as both graph storage and a unified vector index, and a lightweight database providing session-based conversational memory.

IV. EVALUATION SETUP

Evaluation uses 102 test scenarios (fixtures) spread across eight question categories: *conceptual* (25), *yaml_gen* (25), *relationship* (18), *followup* (12), *realworld* (9), *planning* (5), *troubleshooting* (5), and *command* (3). These scenarios were constructed from interviews with three DevOps/SRE practitioners who actively use Kubernetes and LLMs. The three practitioners also rated the realism of sample fixtures, confirming that scenarios grounded in concrete operational situations were judged more representative than purely definitional questions.

System performance is measured along three dimensions. *Answer Quality* (AnsQ) is the average of the answer's semantic similarity to the ground truth, YAML syntactic validity, and schema compliance. *Retrieval Quality* (RetQ) is operationalized as an F1 score between the set of nodes on the system's reasoning path and the set of relevant ground-truth nodes, following the standard Precision/Recall definitions [21]:

$$\mathrm{RetQ}_{F1} = \frac{2 \cdot P \cdot R}{P + R} \tag{1}$$

where P and R denote the Precision and Recall of the reasoning-path nodes against the relevant ground-truth nodes, respectively. *Reasoning Quality* (ReaQ) has two components: *hop accuracy*, the proportion of ground-truth edges successfully traversed by the system,

$$\mathrm{HopAcc} = \frac{|E_{\text{traversed}} \cap E_{\text{ground truth}}|}{|E_{\text{ground truth}}|}, \tag{2}$$

and *faithfulness*, the proportion of answer claims that can be traced back to the retrieved context [12].

These three dimensions are measured to compare three systems using the same dataset, the same speaker model (GPT-4o-mini, temperature=0.1), and identical metrics: *Vanilla LLM* (no context), *Vector RAG* (cosine-similarity top-5 without graph traversal), and GraphRAG (the full system). Statistical significance is tested using the Wilcoxon signed-rank test with a paired bootstrap of 1,000 iterations and Holm-Bonferroni correction for multiple comparisons.

In addition to these quantitative metrics, four independent Kubernetes practitioners rated GraphRAG's answers on three representative scenarios (YAML manifest generation, troubleshooting a failing Pod, and explaining a resource relationship) on a 1–5 scale covering relevance, technical accuracy, completeness, and willingness to use the answer in practice. The system's retrieval trace received the highest average confidence rating among all rated items (4.50 out of 5), and the relationship-explanation scenario received the highest average score across all four dimensions (4.38). This qualitative check complements, rather than replaces, the quantitative evaluation; its purpose is to confirm that the automated metrics track something practitioners actually value.

V. RESULTS AND DISCUSSION

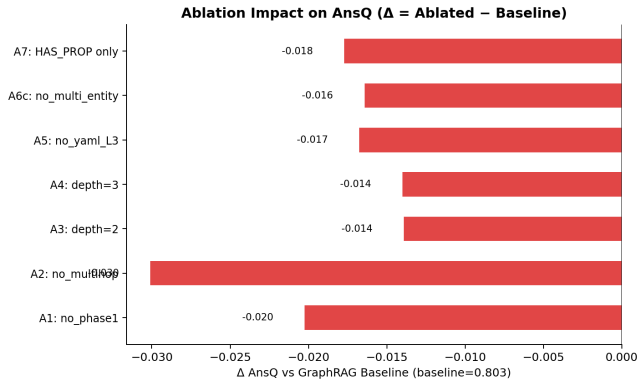

FIGURE 2. Drop in RetQ F1 and hop accuracy for each ablation configuration relative to the baseline.

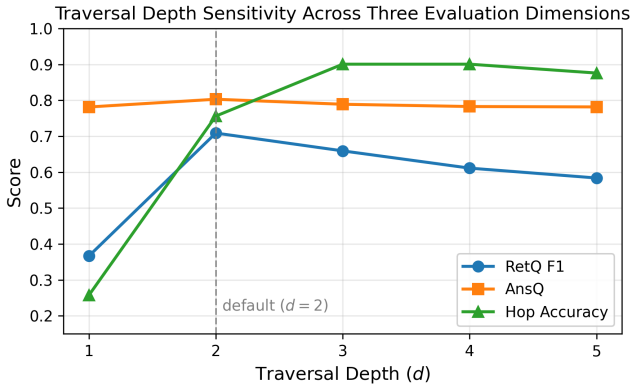

FIGURE 3. RetQ F1, AnsQ, and hop accuracy as a function of traversal depth d.

A. THREE-SYSTEM COMPARISON

Table 1 summarizes the three systems' scores across all evaluation dimensions together with their statistical significance. RetQ is the most discriminative dimension: GraphRAG reaches an F1 of 0.71 versus 0.24 for Vector RAG and 0.00 for Vanilla LLM, both differences significant ($p<0.001$). This advantage stems from multi-hop traversal reaching intermediate structural nodes (e.g., the *Deployment* → *DeploymentSpec* → *PodTemplateSpec* chain) that Vector RAG cannot access. In contrast, AnsQ is comparable across all three systems, and the difference between GraphRAG and Vector RAG is not statistically significant ($p_{\text{Holm}} = 0.067$), indicating that answer text quality is determined by the generative capability of the LLM, which is identical across all three systems, rather than by the retrieval mechanism.

B. COMPONENT CONTRIBUTIONS

Table 2 presents the results of a seven-configuration ablation study, while Fig. 2 visualizes the most notable drops in RetQ and hop accuracy. Disabling multi-hop traversal (A2) produces the largest RetQ drop, confirming that this component is the most critical: without traversal, the system obtains only the seed node and lacks the schema-dependency context. Disabling exact match (A1) also significantly lowers RetQ, confirming this stage's role as the gate that ensures seed-node accuracy. Replacing 18 semantic edge types with a single generic type (A7) significantly lowers both RetQ and hop accuracy, demonstrating the relation taxonomy's contribution to retrieval and reasoning quality. In contrast, a fixed depth of $d = 3$ (A4) does not significantly lower RetQ but does increase hop accuracy, showing that adaptive behavior cannot be fully replaced by a single fixed depth: greater depth increases the coverage of traversed edges but lowers RetQ precision because it draws in additional, less-relevant nodes.

C. TRAVERSAL DEPTH SENSITIVITY

Table 3 and Fig. 3 report RetQ, AnsQ, and hop accuracy across traversal depths $d \in \{1, \ldots, 5\}$, holding every other component fixed at its default configuration.

A depth of $d = 2$, the adaptive default for resource-specific intents, achieves the best joint balance of RetQ (0.71) and AnsQ (0.80). At $d = 1$, retrieval is too shallow: RetQ falls sharply to 0.37 because the seed node alone rarely covers the schema dependencies needed to answer the query. Increasing depth beyond 2 improves hop accuracy, which peaks at $d = 3$ (0.90), but RetQ declines monotonically thereafter (0.66 at $d = 3$, 0.61 at $d = 4$, 0.58 at $d = 5$) because deeper traversal increasingly draws in the generic, widely shared node types described in §III, diluting precision without adding query-relevant structure. AnsQ is comparatively insensitive to depth once $d \geq 2$, ranging narrowly from 0.78 to 0.80, consistent with the finding in §V-A that answer quality is governed primarily by the LLM rather than by retrieval. This trade-off between edge recall and retrieval precision is the empirical basis for adapting depth to query intent rather than fixing a single depth for all queries: no single value of d is jointly optimal across RetQ and hop accuracy, and the intents that benefit from deeper traversal (*generate_yaml*, *trace_relationship*, and *planning*) are exactly those assigned $d = 3$ in this system.

D. BOUNDARY CONDITIONS

To identify where GraphRAG's advantage is largest, Table 4 reports the mean RetQ gain over Vector RAG for each fixture category, defined as the per-fixture difference in RetQ between the two systems.

GraphRAG outperforms Vector RAG on all eight categories with no exceptions. The gain ranges from $+0.354$ for *relationship* queries, which already state resource relations explicitly and therefore benefit least from additional traversal, to $+0.664$ for *followup* queries, which depend on multi-turn context spanning several resources and therefore benefit most from multi-hop retrieval. The overall mean gain across all 102 fixtures is $+0.467$.

Two structural factors were tested for correlation with this per-fixture gain using Spearman's ρ. The degree of the primary node in the knowledge graph (its number of incident edges) correlates weakly but significantly with RetQ gain

TABLE 1. Three-System Score Comparison per Evaluation Dimension ($n = 102$)

Metric	Vanilla LLM	Vector RAG	GraphRAG	Δ vs. Vector RAG	Δ vs. Vanilla LLM
AnsQ	0.7469	0.7984	0.8031	+0.005 (n.s.)	+0.056 (***)
RetQ (F1)	N/A	0.2437	0.7089	+0.465 (***)	+0.709 (***)
Hop Accuracy	N/A	N/A	0.7562	N/A	N/A
Faithfulness	N/A	0.1675	0.3055	+0.127 (***)	N/A
Syntactic Validity	0.8333	0.9333	1.0000	+0.036 (n.s.)	+0.143 (*)

N/A = not applicable or not computable (no retrieval or graph traversal for that system/metric pairing). *** $p < 0.001$; * $p < 0.05$; n.s. = not significant (Wilcoxon + paired bootstrap, 1,000 iterations, Holm-Bonferroni correction).

TABLE 2. Ablation Study: Difference from the GraphRAG Baseline ($n = 102$)

Configuration	ΔAnsQ	ΔRetQ	ΔHopAcc
A1 *no_exact_match*	-0.020	-0.309^{***}	-0.340^{***}
A2 *no_multihop*	-0.030	-0.656^{***}	-0.700^{***}
A3 *depth=2 fixed*	-0.014	-0.123^{***}	-0.147^{***}
A4 *depth=3 fixed*	-0.014	-0.050	$+0.145$
A5 *no_yaml_layer3*	-0.017	-0.057^{**}	$+0.004$
A6 *no_multi_entity*	-0.016	-0.011	-0.007
A7 *1 edge type*	-0.018	-0.152^{***}	-0.232^{***}

Δ = Ablated − baseline; negative values indicate the component contributes positively. *** $p < 0.001$; ** $p < 0.01$ (Wilcoxon + bootstrap, $n = 102$); remaining rows not significant.

TABLE 3. Traversal Depth Sensitivity ($n = 102$)

Depth (d)	RetQ F1	AnsQ	Hop Accuracy
1	0.3666	0.7815	0.2574
2 (default)	**0.7089**	**0.8031**	0.7562
3	0.6593	0.7891	**0.9007**
4	0.6113	0.7828	0.9007
5	0.5838	0.7817	0.8760

$d = 2$ is the adaptive default for the *explain* and *followup* intents; $d = 3$ is the adaptive default for *generate_yaml*, *trace_relationship*, and *planning*.

($\rho = +0.245$, $p = 0.013$, $n = 101$): resources with richer schema connectivity, such as *Deployment* and *StatefulSet*, benefit more from multi-hop traversal because more relevant context becomes reachable. In contrast, the number of hops actually retrieved during evaluation shows no significant correlation with gain ($\rho = -0.082$, $p = 0.414$), indicating that traversal depth per fixture is not by itself a strong predictor of GraphRAG's advantage; the advantage instead depends on the structural richness of the resource being queried.

E. DISCUSSION: FAITHFULNESS AND THE LIMITS OF THE ADVANTAGE

GraphRAG's faithfulness (0.31) is significantly higher than Vector RAG's (0.17), yet its absolute value remains well below an ideal threshold. To quantify this gap, all 818 claims extracted from GraphRAG's answers across the full fixture set were classified into four categories: *supported* (258 claims, 31.5%, directly traceable to the retrieved graph context), *modality* (81 claims, 9.9%, content supported by the context but reformulated from a declarative schema statement into a procedural instruction), *partial* (24 claims, 2.9%, combining a context-supported component with LLM-added detail), and *absent* (455 claims, 55.6%, with no traceable reference in the retrieved context). Combining the first two categories, only 41.4% of claims (339 of 818) are traceable to document context, while the majority originate from the LLM's parametric knowledge rather than from the retrieved graph.

TABLE 4. Mean RetQ Gain over Vector RAG per Fixture Category

Category	n	Mean Gain
followup	12	+0.6642
yaml_gen	25	+0.5450
planning	5	+0.5406
command	3	+0.4614
troubleshooting	5	+0.4219
realworld	9	+0.4014
conceptual	25	+0.3870
relationship	18	+0.3540
Overall	**102**	+0.467

Spearman correlation between graph node degree and RetQ gain: $\rho = +0.245$, $p = 0.013$ ($n = 101$), weak but significant. Between hops retrieved and RetQ gain: $\rho = -0.082$, $p = 0.414$ ($n = 102$), not significant.

This is not a system defect but an architectural consequence: the knowledge graph, built from the *definitions* block, only models object-typed inter-resource relations as edges, while scalar attribute values (such as a container name or image) cannot be represented as graph nodes. The system is therefore deliberately designed to be open-book: the LLM is allowed to complete its answer with parametric knowledge when information is unavailable in the graph. Correlation analysis shows that faithfulness correlates weakly and non-significantly with AnsQ, confirming that the two measure different things: AnsQ measures answer correctness and relevance, while faithfulness measures the traceability of claims to the retrieved context.

F. LIMITATIONS

Three limitations further bound the scope of these findings. First, the knowledge graph built is static and represents only schema-structural relations from the *definitions* block; operational relations such as RBAC bindings are reachable in only one direction, and live cluster runtime state is not represented. Second, retrieval is capped at three hops, so queries requiring context beyond this reach silently fall back on the LLM's parametric knowledge. Among all categories, the *relationship* category receives the smallest, though still positive, benefit from GraphRAG. Third, GraphRAG's advantage does not extend to every evaluation dimension: AnsQ is on par with Vector RAG, and faithfulness reflects the

graph's limited coverage of attribute values rather than a failure of the retrieval mechanism itself.

VI. CONCLUSION

This work builds a GraphRAG system with three technical contributions: deterministic knowledge-graph construction from the Kubernetes OpenAPI schema, a retrieval mechanism with traversal depth adapted per query intent, and three-layer, graph-grounded YAML validation as an alternative to a live-cluster dry-run. Evaluation on 102 test scenarios shows a significant GraphRAG advantage in retrieval (F1 0.71 versus 0.24, $\Delta{+}0.465$ over Vector RAG) and structural reasoning, while answer quality remains comparable across systems since it depends on the same underlying LLM. These findings confirm that graph representation's contribution in the Kubernetes configuration domain lies in the traceability and completeness of structural retrieval, not in answer fluency.

ACKNOWLEDGMENT

This manuscript was prepared with LLM assistance to condense text from the first author's undergraduate thesis; all technical claims, data, and experimental results originate from research conducted and validated by the authors.

REFERENCES

[1] The Kubernetes Authors, "Kubernetes concepts," [Online]. Available: https://kubernetes.io/docs/concepts/, 2024, accessed: May 15, 2024.
[2] Cloud Native Computing Foundation, "CNCF annual survey 2023," [Online]. Available: https://www.cncf.io/reports/cncf-annual-survey-2023/, 2023, accessed: Nov. 22, 2025.
[3] Gartner, "Gartner says cloud will be the centerpiece of new digital experiences," [Online]. Available: https://www.gartner.com/en/newsroom/press-releases/2021-11-10-gartner-says-cloud-will-be-the-centerpiece-of-new-digital-experiences, 2021, accessed: Nov. 22, 2025.
[4] J. Soldani, D. A. Tamburri, and W.-J. Van Den Heuvel, "The pains and gains of microservices: A systematic grey literature review," Journal of Systems and Software, vol. 146, pp. 215–232, 2018.
[5] A. Rahman, A. Partho, P. Morrison, and L. Williams, "Security misconfigurations in open source Kubernetes manifests: An empirical study," ACM Transactions on Software Engineering and Methodology, vol. 32, no. 5, 2023.
[6] Z. Liu, A. Nguyen, J. Chen, X. Zhang, Y. Li, and Y. Xiong, "Deep analysis of LLM-based code generation: Correctness, complexity, and security," IEEE Transactions on Software Engineering, pp. 1–20, 2024.
[7] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," ACM Computing Surveys, vol. 55, no. 12, pp. 1–38, 2023.
[8] Y. Wan, Z. Chen, Y. Liu, C. Chen, and M. Packianather, "Empowering LLMs by hybrid retrieval-augmented generation for domain-centric Q&A in smart manufacturing," Advanced Engineering Informatics, vol. 65, p. 103212, 2025.
[9] H. Han, Y. Wang, H. Shomer, K. Guo, J. Ding, Y. Lei, M. Halappanavar, R. A. Rossi, S. Mukherjee, and X. Tang, "Retrieval-augmented generation with graphs (GraphRAG)," arXiv preprint arXiv:2501.00309, 2024.
[10] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," arXiv preprint arXiv:2312.10997, 2024.
[11] A. Moreno-Cediel, E. Garcia-Lopez, A. Garcia-Cabot, and D. De-Fitero-Dominguez, "Optimising retrieval performance in RAG systems: A new growing window semantic chunking strategy to address weak semantic boundaries," Knowledge-Based Systems, vol. 331, p. 114896, 2025.
[12] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated evaluation of retrieval augmented generation," arXiv preprint arXiv:2309.15217, 2023.
[13] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, "Unifying large language models and knowledge graphs: A roadmap," IEEE Transactions on Knowledge and Data Engineering, vol. 36, no. 7, pp. 3580–3599, 2024.
[14] C. Ma, Y. Chen, T. Wu, A. Khan, and H. Wang, "Large language models meet knowledge graphs for question answering: Synthesis and opportunities," arXiv preprint arXiv:2505.20099, 2025.
[15] M. Hofer, D. Obraczka, A. Saeedi, H. Köpcke, and E. Rahm, "Construction of knowledge graphs: Current state and challenges," Information, vol. 15, no. 8, p. 509, 2024.
[16] B. Abu-Salih, "Domain-specific knowledge graphs: A survey," Journal of Network and Computer Applications, vol. 185, p. 103076, 2021.
[17] X. He, Y. Tian, Y. Sun, N. V. Chawla, T. Laurent, Y. LeCun, X. Bresson, and B. Hooi, "G-Retriever: Retrieval-augmented generation for textual graph understanding and question answering," in The Twelfth International Conference on Learning Representations (ICLR), 2024.
[18] Z. Lu, H. Xu, A. Chen, S. Tang, J. Zhang, Y. Feng, W. Pan, and J. Huang, "HSG-RAG: Hierarchical knowledge base construction for embedded system development," ACM Transactions on Design Automation of Electronic Systems, vol. 30, no. 6, pp. 1–21, 2025.
[19] H.-H. Yu, W.-T. Lin, C.-W. Kuan, C.-C. Yang, and K.-M. Liao, "GraphRAG-enhanced dialogue engine for domain-specific question answering: A case study on the civil IoT taiwan platform," Future Internet, vol. 17, no. 9, p. 414, 2025.
[20] R. Cao, H. Zhang, T. Huang, Z. Kang, Y. Zhang, L. Sun, H. Li, Y. Miao, S. Fan, L. Chen, and K. Yu, "NeuSym-RAG: Hybrid neural symbolic retrieval with multiview structuring for PDF question answering," arXiv preprint arXiv:2505.19754, 2025.
[21] C. D. Manning, P. Raghavan, and H. Schütze, Introduction to Information Retrieval. Cambridge, U.K.: Cambridge University Press, 2008.

JIHAN AURELIA is pursuing her bachelor's degree in Information Systems and Technology at the School of Electrical Engineering and Informatics, Institut Teknologi Bandung, Indonesia. Her undergraduate thesis develops a graph-based retrieval-augmented generation system for Kubernetes configuration. Her research interests include knowledge graph construction and retrieval-augmented generation for large language models.